

NLP Text Classification Notebook

Explanation

This document provides a detailed, line-by-line explanation of the provided Jupyter Notebook for NLP Text Classification. The project covers data preprocessing, TF-IDF vectorization, traditional Machine Learning models, Deep Learning with Keras, Large Language Models (LLMs) using Hugging Face, and finally, deployment with Streamlit.

Table of Contents

1. [Introduction and Library Imports](#)
 2. [Data Loading and Initial Exploration](#)
 3. [Text Preprocessing](#)
 4. [Feature Extraction \(TF-IDF\)](#)
 5. [Traditional Machine Learning Models](#)
 6. [Deep Learning Model \(Keras\)](#)
 7. [Large Language Models \(LLMs\) with Hugging Face](#)
 - [Using pipeline for Zero-Shot Classification](#)
 - [Fine-tuning a Pre-trained Model](#)
 8. [Model Saving and Deployment Preparation](#)
-

1. Introduction and Library Imports

This section imports all necessary libraries for data manipulation, visualization, text preprocessing, machine learning, deep learning, and Hugging Face transformers.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import re
import string
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
```

Explanation:

- `numpy (as np)`: A fundamental package for numerical computation in Python, especially for array operations. (مكتبة أساسية للحسابات الرقمية في بايثون، خاصة لعمليات المصفوفات.)
- `pandas (as pd)`: Used for data manipulation and analysis, particularly with DataFrames. (DataFrame تستخدم لمعالجة وتحليل البيانات، خاصة مع هياكل البيانات.)
- `matplotlib.pyplot (as plt)`: A plotting library for creating static, interactive, and animated visualizations in Python. (مكتبة للرسم البياني لإنشاء تصورات ثابتة وتفاعلية.) ومتحركة في بايثون.
- `seaborn (as sns)`: A data visualization library based on matplotlib, providing a high-level interface for drawing attractive and informative statistical graphics. (مكتبة لتصور توفر واجهة عالية المستوى لرسم رسوم بيانية إحصائية، matplotlib البيانات مبنية على جذابة وغنية بالمعلومات.)
- `re`: Provides regular expression operations. Essential for pattern matching and text cleaning. (توفر عمليات التعبيرات العادية. ضرورية لمطابقة الأنماط وتنظيف النصوص.)
- `string`: Contains a collection of string constants and common string operations. (تحتوي على مجموعة من ثوابت السلاسل وعمليات السلاسل الشائعة.)
- `nltk`: Natural Language Toolkit, a leading platform for building Python programs to work with human language data. (مجموعة أدوات معالجة اللغة الطبيعية، منصة رائدة.) لبناء برامج بايثون للتعامل مع بيانات اللغة البشرية.
- `nltk.corpus.stopwords`: Contains a list of common words (stop words) that are often removed during text preprocessing as they usually don't carry significant meaning. (أثناء المعالجة المسبقة للنصوص لأنها عادة لا تحمل معنى كبيراً.)

- `nltk.stem.WordNetLemmatizer` : Used for lemmatization, which reduces words to their base or root form (lemma). (الذي يقلل الكلمات، (lemmatization) تستخدم للتقليل، إلى شكلها الأساسي أو الجذري)

2. Data Loading and Initial Exploration

This section loads the dataset and performs initial checks to understand its structure and identify any missing or duplicate values.

```
df=pd.read_csv(r"D:\thirdyear\semester2\NLP\section\nlp_dataset.csv")
```

Explanation:

- `pd.read_csv(...)` : Reads a CSV (Comma Separated Values) file into a Pandas DataFrame. The `r"..."` prefix indicates a raw string, which is useful for Windows file paths to avoid issues with backslashes. (القيم المفصولة بفواصل CSV تقرأ ملف) تشير إلى سلسلة نصية خام، وهي `r"..."` البادئة. Pandas DataFrame وتحوله إلى (لتجنب مشاكل الشرطات المائلة العكسية Windows مفيدة لمسارات ملفات)
 - **Note:** The path `D:\thirdyear\semester2\NLP\section\nlp_dataset.csv` suggests this notebook was run in a specific local environment. For reproducibility, this path would need to be adjusted. (ملاحظة: المسار يشير إلى أن هذا `D:\thirdyear\semester2\NLP\section\nlp_dataset.csv` الدفتر تم تشغيله في بيئة محلية محددة. لإعادة الإنتاج، سيحتاج هذا المسار إلى التعديل)

```
df
```

Explanation:

- Displaying `df` shows the first and last 5 rows of the DataFrame, along with the number of rows and columns. This gives a quick overview of the loaded data. (عرض `df` بالإضافة إلى عدد الصفوف والأعمدة. DataFrame يظهر أول وآخر 5 صفوف من `df` يعطي هذا نظرة عامة سريعة على البيانات المحملة)

- **Output shows:** The DataFrame has columns `text` , `label` , and `label_name` . There are 7000 rows. (على DataFrame يظهر الإخراج: يحتوي) `text` g `label` g `label_name` . يوجد 7000 صف.)

```
df["label_name"].value_counts()
```

Explanation:

- `df["label_name"].value_counts()` : This counts the occurrences of each unique value in the `label_name` column. This is crucial for understanding the distribution of classes in the dataset. (تحسب هذه الدالة تكرارات كل قيمة فريدة في عمود `label_name` . هذا أمر بالغ الأهمية لفهم توزيع الفئات في مجموعة البيانات `label_name` .)
 - **Output shows:** A list of `label_name` values and their counts, indicating an imbalanced dataset where some classes have more samples than others (e.g., `rec.sport.hockey` has 389, `talk.religion.misc` has 228). (يظهر الإخراج:). عدد تكراراتها، مما يشير إلى مجموعة بيانات غير متوازنة `label_name` قائمة بقيم (حيث تحتوي بعض الفئات على عينات أكثر من غيرها)

```
df["label_name"].value_counts().count()
```

Explanation:

- `.count()` : After `value_counts()` , calling `.count()` gives the total number of unique labels in the `label_name` column. (استدعاء `value_counts()` ، يعطي `label_name` العدد الإجمالي للتصنيفات الفريدة في عمود `.count()` .)
 - **Output shows:** 20 , meaning there are 20 distinct categories in the dataset. (يظهر الإخراج: 20 ، مما يعني وجود 20 فئة مميزة في مجموعة البيانات)

```
df.isna().sum()
```

Explanation:

- `df.isna()` : Returns a boolean DataFrame indicating `True` for NaN (Not a Number) values and `False` otherwise. (منطقية تشير إلى DataFrame تعيد) `True` بخلاف ذلك `False` و (ليست رقمًا) NaN للقيم

- `.sum()` : Sums the `True` values (which are treated as 1) for each column, effectively counting the number of missing values per column. (التي `True` تجمع قيم) (لكل عمود، مما يحسب عدد القيم المفقودة لكل عمود (تعامل على أنها 1).)
 - **Output shows:** `text 0, label 0, label_name 0`, indicating no missing values in any of the columns. (يظهر الإخراج: `text 0, label 0, label_name 0`, مما يشير إلى عدم وجود قيم مفقودة في أي من الأعمدة، 0.)

```
df.duplicated().sum()
```

Explanation:

- `df.duplicated()` : Returns a boolean Series indicating duplicate rows. (تعيد سلسلة منطقية تشير إلى الصفوف المكررة.)
- `.sum()` : Counts the number of `True` values, which represents the total number of duplicate rows. (والتي تمثل العدد الإجمالي للصفوف المكررة، `True` تحسب عدد قيم) (يظهر الإخراج: 0, indicating no duplicate rows in the dataset. (0, مما يشير إلى عدم وجود صفوف مكررة في مجموعة البيانات).

3. Text Preprocessing

This section defines and applies a text cleaning function to prepare the `text` data for model training. This involves lowercasing, removing non-alphabetic characters, removing stop words, and lemmatization.

```

import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

# Download NLTK resources if not already downloaded
try:
    nltk.data.find('corpora/stopwords')
except nltk.downloader.DownloadError:
    nltk.download('stopwords')
try:
    nltk.data.find('corpora/wordnet')
except nltk.downloader.DownloadError:
    nltk.download('wordnet')

stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def clean_text(text):
    text = text.lower()
    text = re.sub(r'^a-zA-Z', ' ', text)
    words = text.split()
    words = [
        lemmatizer.lemmatize(word)
        for word in words
        if word not in stop_words and len(word) > 2
    ]
    return ' '.join(words)

```

Explanation:

- **NLTK Downloads:** The `try-except` blocks ensure that the necessary NLTK resources (`stopwords` and `wordnet`) are downloaded if they are not already present. This is good practice for reproducibility. (تنزيل موارد `try-except` تضمن كتمل) إذا لم تكن موجودة بالفعل. هذه ممارسة (`stopwords` و `wordnet`) الضرورية NLTK (جيدة لإعادة الإنتاج).
- `stop_words = set(stopwords.words('english'))` : Loads a set of common English stop words (e.g.,) تم التعامل معه ضمناً أو إعادة `label_encoded` لأن عمود (`e.g.,`) أو لم يكن موجوداً في مجموعة البيانات في تلك المرحلة. `set_format` تسميته بواسطة `labels`` بشكل صحيح إلى `label_encoded` بتعيين عمود `set_format` يجب أن يقوم استدعاء (إذا كان موجوداً).

```

from transformers import AutoModelForSequenceClassification,
TrainingArguments, Trainer
import numpy as np
import evaluate

num_labels = len(encoder.classes_)
model = AutoModelForSequenceClassification.from_pretrained(
    "distilbert-base-uncased", num_labels=num_labels
)

def compute_metrics(eval_pred):
    metrics = evaluate.load("accuracy")
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)
    return metrics.compute(predictions=predictions, references=labels)

training_args = TrainingArguments(
    output_dir="./results",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=3,
    weight_decay=0.01,
    evaluation_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
    metric_for_best_model="accuracy",
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    tokenizer=tokenizer,
    compute_metrics=compute_metrics,
)

trainer.train()

```

Explanation:

- `AutoModelForSequenceClassification.from_pretrained("distilbert-base-uncased", num_labels=num_labels)`: Loads a pre-trained DistilBERT model

مدربًا مسبقًا ومناسبًا DistilBERT يحمل نموذج). (للتصنيف التسلسلات

- `"distilbert-base-uncased"` : Specifies the base model to load. (يحدد النموذج الأساسي للتحميل.)
- `num_labels=num_labels` : Configures the output layer of the model to match the number of unique classes in the dataset. (يقوم بتكوين طبقة الإخراج للنموذج). (لتناسب مع عدد الفئات الفريدة في مجموعة البيانات)
- `compute_metrics(eval_pred)` function:
 - This function is used to calculate evaluation metrics during training. (تستخدم هذه الدالة لحساب مقاييس التقييم أثناء التدريب.)
 - `evaluate.load("accuracy")` : Loads the accuracy metric from the `evaluate` library. (يحمل مقياس الدقة من مكتبة `evaluate`.)
 - `logits, labels = eval_pred` : Unpacks the predictions (logits) and true labels from the evaluation prediction object. (logits) يفك حزمة التنبؤات). (والتصنيفات الحقيقية من كائن تنبؤ التقييم)
 - `predictions = np.argmax(logits, axis=-1)` : Converts the logits (raw output from the model) into class predictions by taking the index of the maximum value along the last axis. (الإخراج الخام من النموذج) logits يحول إلى). (تنبؤات فئة عن طريق أخذ فهرس القيمة القصوى على طول المحور الأخير)
 - `return metrics.compute(predictions=predictions, references=labels)` : Computes and returns the accuracy. (يحسب ويعيد الدقة.)
- `TrainingArguments(...)` : Defines the hyperparameters and settings for training the model. (يحدد المعلمات الفائقة والإعدادات لتدريب النموذج).
 - `output_dir="./results"` : Directory where model checkpoints and predictions will be saved. (الدليل الذي سيتم فيه حفظ نقاط فحص النموذج). (والتنبؤات)
 - `learning_rate=2e-5` : The initial learning rate for the optimizer. (معدل التعلم الأولي للمُحسِّن.)
 - `per_device_train_batch_size=16` , `per_device_eval_batch_size=16` : Batch sizes for training and evaluation. (أحجام الدفعات للتدريب والتقييم.)
 - `num_train_epochs=3` : Total number of training epochs to perform. (العدد). (الإجمالي لعصور التدريب التي سيتم إجراؤها)

- `weight_decay=0.01` : The weight decay to apply (if not zero, it's a form of L2 regularization). (إذا لم يكن صفرًا، فهو شكل من تضاؤل الوزن الذي سيتم تطبيقه. (L2 أشكال تنظيم
- `evaluation_strategy="epoch"` : Evaluation is performed at the end of each epoch. (يتم إجراء التقييم في نهاية كل عصر.)
- `save_strategy="epoch"` : Model checkpoints are saved at the end of each epoch. (يتم حفظ نقاط فحص النموذج في نهاية كل عصر.)
- `load_best_model_at_end=True` : Loads the best model (based on `metric_for_best_model`) at the end of training. (بناءً على) يحمل أفضل نموذج (في نهاية التدريب) (`metric_for_best_model`)
- `metric_for_best_model="accuracy"` : The metric to use to compare models and determine the best one. (المقياس الذي سيستخدم لمقارنة النماذج وتحديد) (الأفضل.)
- `Trainer(...)` : Initializes the Hugging Face `Trainer` object, which is a high-level API for training models. (وهو واجهة برمجة، Hugging Face `Trainer` يهيئ كائن) (تطبيقات عالية المستوى لتدريب النماذج)
 - `model`, `args`, `train_dataset`, `eval_dataset`, `tokenizer`, `compute_metrics` : All the components required for training. (جميع المكونات) (المطلوبة للتدريب)
- `trainer.train()` : Starts the training process. (يبدأ عملية التدريب.)

```
trainer.evaluate()
```

Explanation:

- `trainer.evaluate()` : Evaluates the trained model on the `eval_dataset` (which is `test_dataset` in this case) and returns the evaluation metrics, including the loss. (في هذه الحالة `test_dataset` وهو) `eval_dataset` يقيم النموذج المدرب على) (ويعيد مقاييس التقييم، بما في ذلك الخسارة)
 - **Output shows:** `{'eval_loss': 1.2886754274368286}` . This is the validation loss after fine-tuning. (يظهر الإخراج) `{'eval_loss': 1.2886754274368286}` . (هذه هي خسارة التحقق بعد الضبط الدقيق)

8. Model Saving and Deployment Preparation

This section demonstrates how to save the trained traditional machine learning models (SVM and TF-IDF vectorizer) for later use, likely for deployment with Streamlit.

```
import joblib

joblib.dump(svm_model, 'svm_model.pkl')
joblib.dump(vectorizer, 'tfidf.pkl')
```

Explanation:

- `import joblib`: Imports the `joblib` library, which is efficient for serializing and deserializing Python objects, especially large NumPy arrays. (`joblib` مكتبة Python ، الكبيرة NumPy وهي فعالة لتسلسل وإلغاء تسلسل كائنات بايثون، خاصة مصفوفات NumPy.)
- `joblib.dump(svm_model, 'svm_model.pkl')`: Saves the trained `svm_model` to a file named `svm_model.pkl`. This allows the model to be loaded later without retraining. (هذا يسمح بـ `svm_model.pkl` المدرب إلى ملف يسمى `svm_model` بحفظ). (بتحميل النموذج لاحقًا دون إعادة التدريب.)
- `joblib.dump(vectorizer, 'tfidf.pkl')`: Saves the fitted `TfidfVectorizer` to a file named `tfidf.pkl`. This is crucial because the same vectorizer used for training must be used for new data during inference. (`TfidfVectorizer` يحفظ). (vectorizer هذا أمر بالغ الأهمية لأن نفس الـ `tfidf.pkl` المناسب إلى ملف يسمى `tfidf` المستخدم للتدريب يجب أن يستخدم للبيانات الجديدة أثناء الاستدلال.)

```
model = joblib.load('svm_model.pkl')
vectorizer = joblib.load('tfidf.pkl')
```

Explanation:

- `joblib.load('svm_model.pkl')`: Loads the saved SVM model from the `svm_model.pkl` file. (SVM نموذج يحمل من ملف `svm_model.pkl` المحفوظ.)
- `joblib.load('tfidf.pkl')`: Loads the saved TF-IDF vectorizer from the `tfidf.pkl` file. (TF-IDF vectorizer يحمل من ملف `tfidf.pkl` المحفوظ.)

These loaded objects (`model` and `vectorizer`) would then be used in a Streamlit application to preprocess new input text and make predictions. (ستستخدم هذه الكائنات) لمعالجة النص المدخل الجديد Streamlit بعد ذلك في تطبيق (`model` و `vectorizer`) المحملة (.وإجراء التنبؤات)

the, a, an, is, are, etc.) that are commonly removed to focus on more meaningful words. (إلخ the, a, an, is, are, مثل) تحمل مجموعة من كلمات التوقف الإنجليزية الشائعة التي يتم (إزالتها عادة للتركيز على الكلمات الأكثر أهمية).

- `lemmatizer = WordNetLemmatizer()` : Initializes the WordNet Lemmatizer. (يهيئ WordNet Lemmatizer.)
- `clean_text(text)` function:
 - `text = text.lower()` : Converts all characters in the text to lowercase to ensure uniformity and prevent the same word from being treated as different due to capitalization. (يحول جميع الأحرف في النص إلى أحرف صغيرة لضمان.) (التوحيد ومنع معاملة نفس الكلمة على أنها مختلفة بسبب الأحرف الكبيرة)
 - `text = re.sub(r'[^\a-zA-Z]', ' ', text)` : Uses regular expressions to remove any character that is not an English alphabet letter (a-z, A-Z) and replaces it with a space. This helps in removing punctuation, numbers, and special symbols. (يستخدم التعبيرات العادية لإزالة أي حرف ليس حرفًا أبجديًا إنجليزيًا.) (يستبدله بمسافة. يساعد هذا في إزالة علامات الترقيم والأرقام والرموز (a-z, A-Z) الخاصة.)
 - `words = text.split()` : Splits the cleaned text into a list of individual words. (يقسم النص المنظف إلى قائمة من الكلمات الفردية.)
 - `words = [lemmatizer.lemmatize(word) for word in words if word not in stop_words and len(word) > 2]` : This is a list comprehension that performs two main tasks:
 - **Lemmatization:** `lemmatizer.lemmatize(word)` reduces each word to its base form (e.g., “running” becomes “run”, “better” becomes “good”). (التقليم: يقلل كل كلمة إلى شكلها الأساسي.)
 - **Stop Word Removal:** `if word not in stop_words` filters out common words that typically do not contribute much to the meaning of a sentence. (إزالة كلمات التوقف: يقوم بتصفية الكلمات الشائعة التي لا تساهم كثيرًا في معنى الجملة.)

- **Short Word Removal:** `and len(word) > 2` removes words with two or fewer characters, which are often noise or insignificant after preprocessing. (إزالة الكلمات القصيرة: يزيل الكلمات التي تحتوي على حرفين.) (أو أقل، والتي غالبًا ما تكون ضوضاء أو غير مهمة بعد المعالجة المسبقة.)
- `return ' '.join(words)` : Joins the processed words back into a single string, separated by spaces. (يعيد الكلمات المعالجة إلى سلسلة نصية واحدة، مفصولة بمسافات.)

```
df['clean_text'] = df['text'].apply(clean_text)
print(df[['text', 'clean_text']].head())
```

Explanation:

- `df['clean_text'] = df['text'].apply(clean_text)` : Applies the `clean_text` function to each entry in the `text` column of the DataFrame and stores the result in a new column called `clean_text`. (على كل إدخال في عمود `clean_text` يطبق دالة.) (`clean_text` ويخزن النتيجة في عمود جديد يسمى `DataFrame` من `text`.)
- `print(df[['text', 'clean_text']].head())` : Displays the first few rows of the original `text` and the newly created `clean_text` columns to show the effect of the preprocessing. (الأصلي و `text` يعرض الصفوف القليلة الأولى من عمودي.) (الذي تم إنشاؤه حديثًا لإظهار تأثير المعالجة المسبقة `clean_text`.)

4. Feature Extraction (TF-IDF)

This section prepares the data for machine learning models by splitting it into training and testing sets and then converting the text data into numerical features using TF-IDF.

```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer

X = df['clean_text']
y = df['label_name']

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Explanation:

- `X = df['clean_text']` : Assigns the preprocessed text data (features) to variable `X`. (`X` يسند بيانات النص المعالجة مسبقًا (الميزات) إلى المتغير `X`.)
- `y = df['label_name']` : Assigns the target labels to variable `y`. (يسند التصنيفات `y`. `y` المستهدفة إلى المتغير `y`.)
- `train_test_split(...)` : Splits the dataset into training and testing sets.
 - `test_size=0.2` : 20% of the data will be used for testing, and 80% for training. (من البيانات ستستخدم للاختبار، و 80% للتدريب 20%.)
 - `random_state=42` : Ensures reproducibility of the split. (يضمن قابلية إعادة إنتاج. (التقسيم.)
 - `stratify=y` : Ensures that the proportion of classes in the training and testing sets is the same as in the original dataset, which is important for imbalanced datasets. (يضمن أن نسبة الفئات في مجموعات التدريب والاختبار هي نفسها في. (مجموعة البيانات الأصلية، وهو أمر مهم لمجموعات البيانات غير المتوازنة.)

```
vectorizer = TfidfVectorizer(  
    max_features=30000,  
    ngram_range=(1,3),  
    min_df=2,  
    max_df=0.9,  
    sublinear_tf=True  
)  
  
X_train_vec = vectorizer.fit_transform(X_train)  
X_test_vec = vectorizer.transform(X_test)
```

Explanation:

- `TfidfVectorizer(...)` : Initializes a TF-IDF (Term Frequency-Inverse Document Frequency) vectorizer.
 - **TF-IDF**: A numerical statistic that is intended to reflect how important a word is to a document in a collection or corpus. (إحصائية عددية تهدف إلى عكس مدى أهمية الكلمة لمستند في مجموعة أو مجموعة نصوص)
 - `max_features=30000` : Limits the number of features (unique words/n-grams) to the top 30,000 most frequent ones. (n-grams الكلمات/ال) يحد من عدد الميزات (إلى أفضل 30,000 الأكثر تكرارًا (الفريدة))
 - `ngram_range=(1, 3)` : Considers unigrams (single words), bigrams (two-word phrases), and trigrams (three-word phrases) as features. (يأخذ في الاعتبار) والعبارات (bigrams)، والعبارات المكونة من كلمتين (unigrams) الكلمات الفردية (كميزات) (trigrams) المكونة من ثلاث كلمات
 - `min_df=2` : Ignores terms that have a document frequency strictly lower than the given threshold (i.e., appear in fewer than 2 documents). (يتجاهل) المصطلحات التي يكون تكرارها في المستندات أقل تمامًا من العتبة المحددة (أي تظهر في أقل من مستندين)
 - `max_df=0.9` : Ignores terms that have a document frequency strictly higher than the given threshold (i.e., appear in more than 90% of the documents). This helps remove very common words that might not be useful discriminators. (يتجاهل المصطلحات التي يكون تكرارها في المستندات أعلى تمامًا من العتبة المحددة (أي تظهر في أكثر من 90% من المستندات). يساعد هذا في إزالة الكلمات الشائعة جدًا التي قد لا تكون مميزة)

- `sublinear_tf=True` : Applies sublinear TF scaling, i.e., `tf` is replaced with $1 + \log(tf)$. This dampens the effect of very frequent terms. (TF يطبق تحجيم هذا يقلل من تأثير المصطلحات. $1 + \log(tf)$ بـ `tf` شبه الخطي، أي يتم استبدال المتكررة جدًا.)
- `X_train_vec = vectorizer.fit_transform(X_train)` : Learns the vocabulary and IDF from the training data (`X_train`) and then transforms the training text into TF-IDF feature vectors. (ثم يحول (`X_train`) من بيانات التدريب IDF يتعلم المفردات و) TF-IDF نص التدريب إلى متجهات ميزات
- `X_test_vec = vectorizer.transform(X_test)` : Transforms the testing text data (`X_test`) into TF-IDF feature vectors using the vocabulary and IDF learned from the training data. It's crucial *not* to `fit` on the test data to prevent data leakage. (يحول IDF باستخدام المفردات و TF-IDF إلى متجهات ميزات (`X_test`) بيانات نص الاختبار على بيانات الاختبار لمنع تسرب `fit` الاستفادة من بيانات التدريب. من الأهمية بمكان عدم البيانات.)

5. Traditional Machine Learning Models

This section trains and evaluates three traditional machine learning models: Logistic Regression, Multinomial Naive Bayes, and Linear Support Vector Machine (SVM).

```
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import LinearSVC
from sklearn.metrics import classification_report, confusion_matrix,
accuracy_score

# Logistic Regression
lr_model = LogisticRegression(max_iter=1000, solver='liblinear')
lr_model.fit(X_train_vec, y_train)
lr_pred = lr_model.predict(X_test_vec)
print("Logistic Regression Accuracy:")
print(accuracy_score(y_test, lr_pred))
print(classification_report(y_test, lr_pred))
```

Explanation:

- `LogisticRegression(...)` : Initializes a Logistic Regression model.
 - `max_iter=1000` : Sets the maximum number of iterations for the solver to converge. (يحدد الحد الأقصى لعدد التكرارات للمحلل للتقارب.)
 - `solver='liblinear'` : Specifies the algorithm to use in the optimization problem. `liblinear` is a good choice for small datasets and supports L1 and L2 regularization. (يحدد الخوارزمية التي ستستخدم في مشكلة التحسين). `liblinear` خيار جيد لمجموعات البيانات الصغيرة ويدعم تنظيم L1 و L2.)
- `lr_model.fit(X_train_vec, y_train)` : Trains the Logistic Regression model using the TF-IDF vectorized training data (`X_train_vec`) and corresponding training labels (`y_train`). (يدرب نموذج الانحدار اللوجستي باستخدام بيانات التدريب الموجهة بـ TF-IDF (`X_train_vec`) وتصنيفات التدريب المقابلة (`y_train`).)
- `lr_pred = lr_model.predict(X_test_vec)` : Makes predictions on the TF-IDF vectorized test data (`X_test_vec`). (يقوم بإجراء تنبؤات على بيانات الاختبار الموجهة بـ TF-IDF (`X_test_vec`).)
- `accuracy_score(y_test, lr_pred)` : Calculates the accuracy of the model by comparing the true test labels (`y_test`) with the predicted labels (`lr_pred`). (يحسب مع التصنيفات المتوقعة (`y_test`) دقة النموذج بمقارنة تصنيفات الاختبار الحقيقية (`lr_pred`).)
- `classification_report(y_test, lr_pred)` : Generates a text report showing the main classification metrics (precision, recall, f1-score) for each class, along with support. (f1 الدقة، الاستدعاء، درجة) ينشئ تقريرًا نصيًا يوضح مقاييس التصنيف الرئيسية. (لكل فئة، بالإضافة إلى الدعم)
 - **Output shows:** An accuracy of `0.865` for Logistic Regression. (يظهر الإخراج: دقة `0.865` للانحدار اللوجستي)

```
# Multinomial Naive Bayes
nb_model = MultinomialNB()
nb_model.fit(X_train_vec, y_train)
nb_pred = nb_model.predict(X_test_vec)
print("Naive Bayes Accuracy:")
print(accuracy_score(y_test, nb_pred))
print(classification_report(y_test, nb_pred))
```

Explanation:

- `MultinomialNB()` : Initializes a Multinomial Naive Bayes classifier, which is well-suited for classification with discrete features (like word counts or TF-IDF values). وهو مناسب تمامًا للتصنيف باستخدام الميزات (Multinomial Naive Bayes، يهيئ مصنف) (TF-IDF). مثل عدد الكلمات أو قيم) المنفصلة
- `nb_model.fit(X_train_vec, y_train)` : Trains the Naive Bayes model. (يدرب Naive Bayes نموذج)
- `nb_pred = nb_model.predict(X_test_vec)` : Makes predictions on the test data. (يقوم بإجراء تنبؤات على بيانات الاختبار)
- **Output shows:** An accuracy of `0.8329` for Naive Bayes. (يظهر الإخراج: دقة) `0.8329` Naive Bayes.)

```
# Linear Support Vector Machine (SVM)
svm_model = LinearSVC(
    C=3,
    class_weight='balanced',
    max_iter=10000
)
svm_model.fit(X_train_vec, y_train)
svm_pred = svm_model.predict(X_test_vec)
print("Linear SVM Accuracy:")
print(accuracy_score(y_test, svm_pred))
print(classification_report(y_test, svm_pred))
```

Explanation:

- `LinearSVC(...)` : Initializes a Linear Support Vector Classifier.
 - `C=3` : Regularization parameter. The strength of the regularization is inversely proportional to C. Must be strictly positive. (معامل التنظيم. قوة التنظيم تتناسب) (يجب أن تكون موجبة تمامًا. C عكسياً مع)
 - `class_weight='balanced'` : Automatically adjusts weights inversely proportional to class frequencies in the input data. This is useful for handling imbalanced datasets. (ي ضبط الأوزان تلقائيًا بشكل يتناسب عكسياً مع تكرارات) (الفئات في بيانات الإدخال. هذا مفيد للتعامل مع مجموعات البيانات غير المتوازنة)
 - `max_iter=10000` : Sets the maximum number of iterations to be run. (يحدد الحد) (الأكصى لعدد التكرارات التي سيتم تشغيلها)

- `svm_model.fit(X_train_vec, y_train)` : Trains the Linear SVM model. (يدرب Linear SVM نموذج.)
- `svm_pred = svm_model.predict(X_test_vec)` : Makes predictions on the test data. (يقوم بإجراء تنبؤات على بيانات الاختبار.)
- **Output shows:** An accuracy of `0.89` for Linear SVM. (يظهر الإخراج: دقة `0.89` لـ Linear SVM.)

```
results = {
    'Logistic Regression': accuracy_score(y_test, lr_pred),
    'Naive Bayes': accuracy_score(y_test, nb_pred),
    'Linear SVM': accuracy_score(y_test, svm_pred)
}

for model, score in results.items():
    print(f"{model}: {score:.4f}")

best_model = max(results, key=results.get)
print("Best Model:", best_model)
print("Best Accuracy:", results[best_model])
```

Explanation:

- This code block summarizes the accuracy scores of the three traditional ML models and identifies the best-performing one. (يلخص هذا الجزء من الكود درجات الدقة لنماذج) (التعلم الآلي التقليدية الثلاثة ويحدد الأفضل أداءً.)
- **Output shows:** Linear SVM as the best model with an accuracy of `0.89`. (يظهر) . `0.89` Linear SVM كأفضل نموذج بدقة الإخراج: Linear SVM `0.89` بدقة.)

```
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

cm = confusion_matrix(y_test, svm_pred)

plt.figure(figsize=(12,10))
sns.heatmap(cm, cmap='Blues')
plt.title('Confusion Matrix')
plt.show()
```

Explanation:

- `confusion_matrix(y_test, svm_pred)` : Computes the confusion matrix to evaluate the accuracy of a classification. Each row represents the instances in an actual class, while each column represents the instances in a predicted class. (يحسب مصفوفة الارتباك لتقييم دقة التصنيف. يمثل كل صف الحالات في فئة فعلية، بينما يمثل كل عمود الحالات في فئة متوقعة.)
 - `sns.heatmap(cm, cmap='Blues')` : Visualizes the confusion matrix using a heatmap from the Seaborn library. The `cmap='Blues'` argument sets the color scheme. (الوسيط Seaborn. يصور مصفوفة الارتباك باستخدام خريطة حرارية من مكتبة `cmap='Blues'` يحدد نظام الألوان.)
 - `plt.title('Confusion Matrix')` and `plt.show()` : Sets the title of the plot and displays it. (يحدد عنوان الرسم البياني ويعرضه.)
-

6. Deep Learning Model (Keras)

This section builds and trains a simple feed-forward neural network using Keras for text classification.

```
X_train_dl = X_train_vec.toarray()
X_test_dl = X_test_vec.toarray()
```

Explanation:

- `X_train_vec.toarray()` and `X_test_vec.toarray()` : Converts the sparse TF-IDF matrices (output of `TfidfVectorizer`) into dense NumPy arrays. Keras/TensorFlow models typically expect dense input arrays. (يحول مصفوفات TF-IDF كثيفة. تتوقع نماذج NumPy إلى مصفوفات `TfidfVectorizer` إخراج) المتفرقة Keras/TensorFlow عادةً مصفوفات إدخال كثيفة.)

```
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()

y_encoded = encoder.fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y_encoded,
    test_size=0.2,
    random_state=42,
    stratify=y_encoded
)
```

Explanation:

- `LabelEncoder()` : Initializes a `LabelEncoder` to convert categorical text labels (`label_name`) into numerical labels (integers). This is necessary for Keras models, which require numerical targets. (لتحويل التصنيفات النصية `LabelEncoder` يهيئ). هذا ضروري لنماذج (`label_name`) الفئوية Keras, إلى تصنيفات رقمية (أعداد صحيحة). (التي تتطلب أهدافًا رقمية).
- `y_encoded = encoder.fit_transform(y)` : Fits the encoder to the `y` (original `label_name`) and transforms them into numerical representations. (يناسب الـ `encoder` لـ `y` (الـ `label_name` الأصلي) إلى تمثيلات رقمية (الأصلي `label_name`)).
- The `train_test_split` is performed again, but this time using the `y_encoded` labels, ensuring that the target variable for the DL model is in the correct format. (يتم مرة أخرى، ولكن هذه المرة باستخدام التصنيفات `train_test_split` إجراء `y_encoded`, في التنسيق الصحيح DL مما يضمن أن المتغير الهدف لنموذج).

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout

model = Sequential()
model.add(Dense(
    256,
    activation='relu',
    input_shape=(X_train_dl.shape[1],)
))
model.add(Dropout(0.3))
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(len(np.unique(y_train)), activation='softmax'))

```

Explanation:

- `Sequential()` : Initializes a sequential Keras model, which is a linear stack of layers. (تسلسلي، وهو مكدس خطي من الطبقات Keras يهيئ نموذج.)
- `model.add(Dense(256, activation='relu', input_shape=(X_train_dl.shape[1],)))` : Adds the first dense (fully connected) layer with 256 neurons and ReLU (Rectified Linear Unit) activation function. `input_shape` specifies the shape of the input data, which is the number of features from the TF-IDF vectorizer. (يضيف الطبقة الكثيفة الأولى (المتصلة بالكامل) مع 256 عصبوناً ودالة) `ReLU` تنشيط `input_shape` يحدد من عدد الميزات من `TF-IDF` شكل بيانات الإدخال، وهو عدد الميزات من `TF-IDF` vectorizer.)
- `model.add(Dropout(0.3))` : Adds a dropout layer with a dropout rate of 0.3. Dropout is a regularization technique that randomly sets a fraction of input units to 0 at each update during training time, which helps prevent overfitting. (يضيف طبقة) تسرب بمعدل تسرب 0.3. التسرب هو تقنية تنظيم تعمل على تعيين جزء عشوائي من وحدات الإدخال إلى 0 عند كل تحديث أثناء وقت التدريب، مما يساعد على منع التجاوز.)
- `model.add(Dense(128, activation='relu'))` : Adds another dense layer with 128 neurons and ReLU activation. (يضيف طبقة كثيفة أخرى مع 128 عصبوناً وتنشيط) `ReLU`.)
- `model.add(Dense(len(np.unique(y_train)), activation='softmax'))` : Adds the output layer. The number of neurons is equal to the number of unique classes (`len(np.unique(y_train))`). The `softmax` activation function is used for multi-class classification, outputting probabilities for each class. (يضيف طبقة الإخراج. عدد.)

للتصنيف متعدد softmax العصبونات يساوي عدد الفئات الفريدة. تستخدم دالة تنشيط (الفئات، وتخرج احتمالات لكل فئة)

```
model.compile(  
    optimizer='adam',  
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)
```

Explanation:

- `model.compile(...)` : Configures the model for training.
 - `optimizer='adam'` : Specifies the Adam optimizer, a popular choice for deep learning due to its efficiency. (وهو خيار شائع للتعليم العميق، Adam يحدد مُحسَّن) (بسبب كفاءته)
 - `loss='sparse_categorical_crossentropy'` : Sets the loss function. This is suitable for multi-class classification problems where the labels are integers (not one-hot encoded). (يحدد دالة الخسارة. هذا مناسب لمشاكل التصنيف متعدد) (الفئات حيث تكون التصنيفات أعدادًا صحيحة)
 - `metrics=['accuracy']` : Specifies the metrics to be evaluated by the model during training and testing. (يحدد المقاييس التي سيتم تقييمها بواسطة النموذج) (أثناء التدريب والاختبار)

```
from tensorflow.keras.callbacks import EarlyStopping  
  
early_stop = EarlyStopping(  
    monitor='val_loss',  
    patience=3,  
    restore_best_weights=True  
)
```

Explanation:

- `EarlyStopping(...)` : Initializes an `EarlyStopping` callback. This callback is used to stop training when a monitored quantity has stopped improving. (يستخدم رد الاتصال هذا لإيقاف التدريب عندما تتوقف كمية EarlyStopping اتصال) (مراقبة عن التحسن)

- `monitor='val_loss'` : The quantity to be monitored is the validation loss. (الكمية التي سيتم مراقبتها هي خسارة التحقق)
- `patience=3` : Training will stop if the validation loss does not improve for 3 consecutive epochs. (سيتم إيقاف التدريب إذا لم تتحسن خسارة التحقق لمدة 3 عصور متتالية)
- `restore_best_weights=True` : Restores model weights from the epoch with the best value of the monitored quantity. (يستعيد أوزان النموذج من العصر الذي يحتوي على أفضل قيمة للكمية المراقبة)

```
y_train_encoded = encoder.fit_transform(y_train)
y_test_encoded = encoder.transform(y_test)

history = model.fit(
    X_train_dl,
    y_train_encoded,
    validation_split=0.2,
    epochs=20,
    batch_size=32,
    callbacks=[early_stop]
)
```

Explanation:

- `y_train_encoded = encoder.fit_transform(y_train)` and `y_test_encoded = encoder.transform(y_test)` : Re-encodes the `y_train` and `y_test` labels. It seems there was a previous encoding, and this step ensures the labels are correctly encoded for the DL model training. (يبدو أنه `y_train` و `y_test` يعيد ترميز تصنيفات). كان هناك ترميز سابق، وتضمن هذه الخطوة أن التصنيفات مرمزة بشكل صحيح لتدريب DL نموذج.)
- `model.fit(...)` : Trains the neural network model.
 - `X_train_dl, y_train_encoded` : Training data and labels.
 - `validation_split=0.2` : Sets aside 20% of the training data as validation data to monitor performance during training. (يخصص 20% من بيانات التدريب كبيانات تحقق لمراقبة الأداء أثناء التدريب)
 - `epochs=20` : Maximum number of training epochs. Early stopping will likely stop it sooner. (الحد الأقصى لعدد عصور التدريب. من المرجح أن يوقفه التوقف المبكر.)

- `batch_size=32` : Number of samples per gradient update. (عدد العينات لكل تحديث تدرج.)
- `callbacks=[early_stop]` : Integrates the `early_stop` callback to prevent overfitting. (لمنع التجاوز `early_stop` يدمج رد الاتصال.)

```
loss, accuracy = model.evaluate(X_test_dl, y_test_encoded)
print("DL Model Accuracy:", accuracy)
```

Explanation:

- `model.evaluate(X_test_dl, y_test_encoded)` : Evaluates the trained deep learning model on the test set. (يقيم نموذج التعلم العميق المدرب على مجموعة الاختبار.)
- **Output shows:** A DL Model Accuracy of `0.8828` . (تبلغ DL يظهر الإخراج: دقة نموذج . `0.8828` .)

7. Large Language Models (LLMs) with Hugging Face

This section explores using Hugging Face Transformers for text classification, first with a pre-trained pipeline and then with fine-tuning a DistilBERT model.

Using `pipeline` for Zero-Shot Classification

```
from transformers import pipeline
classifier = pipeline(
    "text-classification",
    model="distilbert-base-uncased-finetuned-sst-2-english"
)
```

Explanation:

- `from transformers import pipeline` : Imports the `pipeline` function from the Hugging Face `transformers` library. The `pipeline` is a high-level API that abstracts away much of the complexity of using pre-trained models. (يستورد دالة)

pipeline هي واجهة برمجة pipeline الـ transformers من مكتبة Hugging Face pipeline (تطبيقات عالية المستوى تجرد الكثير من تعقيد استخدام النماذج المدربة مسبقًا).

- `classifier = pipeline("text-classification", model="distilbert-base-uncased-finetuned-sst-2-english")` : Initializes a text classification pipeline.
 - `"text-classification"` : Specifies the task to be performed. (يحدد المهمة التي سيتم تنفيذها)
 - `model="distilbert-base-uncased-finetuned-sst-2-english"` : Loads a specific pre-trained model. This model is DistilBERT (a smaller, faster version of BERT) that has been fine-tuned on the SST-2 dataset for sentiment analysis (classifying text as positive or negative). (يحمل نموذجًا محددًا مدربًا مسبقًا. هذا). تم ضبطه بدقة على (BERT نسخة أصغر وأسرع من) DistilBERT النموذج هو (لتحليل المشاعر) تصنيف النص على أنه إيجابي أو سلبي) SST-2 مجموعة بيانات

```
sample_text = df['text'][0]
result = classifier(sample_text)
print(result)
```

Explanation:

- `sample_text = df['text'][0]` : Selects the first text entry from the original `text` column of the DataFrame as a sample for prediction. (يختار إدخال النص الأول من عمود `text` كعينة للتنبؤ DataFrame الأصلي في `text`.)
- `result = classifier(sample_text)` : Passes the `sample_text` to the initialized `classifier` pipeline to get a prediction. (إلى خط أنابيب `sample_text` يمرر `classifier` المهيا للحصول على تنبؤ `classifier`.)
- **Output shows:** `[{'label': 'NEGATIVE', 'score': 0.9986326098442078}]` . This indicates that the pre-trained model classified the sample text as 'NEGATIVE' with a high confidence score. (يظهر الإخراج: `[{'label': 'NEGATIVE', 'score': 0.9986326098442078}]` . يشير هذا إلى أن النموذج المدرب مسبقًا صنف النص العينة . بدرجة ثقة عالية 'NEGATIVE' على أنه

Observation (from notebook markdown):

The pre-trained model was trained for sentiment analysis and did not perform well on our custom dataset categories. Therefore, fine-tuning was required.

Explanation: This observation highlights a common challenge: a model pre-trained for a specific task (sentiment analysis) might not generalize well to a different task (multi-class topic classification) or a different domain without further training. This necessitates fine-tuning. توضح هذه الملاحظة تحديًا شائعًا: قد لا يعمم نموذج مدرب مسبقًا لمهمة معينة (تحليل المشاعر) بشكل جيد على مهمة مختلفة (تصنيف الموضوع متعدد الفئات) أو مجال مختلف (دون تدريب إضافي). هذا يستلزم الضبط الدقيق.

Fine-tuning a Pre-trained Model

This sub-section focuses on fine-tuning a DistilBERT model on the custom dataset for better performance on the specific text classification task.

```
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()

df['label_encoded'] = encoder.fit_transform(df['label_name'])
```

Explanation:

- This is a repeat of the `LabelEncoder` initialization and fitting, ensuring that the `label_name` column is encoded into numerical `label_encoded` for use with Hugging Face models. وتناسبه، مما يضمن ترميز `LabelEncoder` هذا تكرر لتهيئة. (Hugging Face رقمي للاستخدام مع نماذج `label_encoded` إلى `label_name` عمود)

```
from sklearn.model_selection import train_test_split

train_df, test_df = train_test_split(
    df,
    test_size=0.2,
    random_state=42,
    stratify=df['label_encoded']
)
```

Explanation:

- The dataset is split into training and testing DataFrames (`train_df` , `test_df`) using `train_test_split` , similar to the earlier split, but now using the

label_encoded column for stratification. (يتم تقسيم مجموعة البيانات إلى train_df , test_df باستخدام train_test_split , للتقسيم label_encoded على غرار التقسيم السابق، ولكن الآن باستخدام عمود الطبقي.)

```
from datasets import Dataset

train_dataset = Dataset.from_pandas(
    train_df[['text', 'label_encoded']]
)

test_dataset = Dataset.from_pandas(
    test_df[['text', 'label_encoded']]
)
```

Explanation:

- `from datasets import Dataset`: Imports the `Dataset` class from the Hugging Face `datasets` library. This library provides efficient data structures for NLP tasks. توفر هذه المكتبة هيكل . datasets من مكتبة Dataset يستورد فئة (بيانات فعالة لمهام معالجة اللغة الطبيعية)
- `Dataset.from_pandas(...)`: Converts the Pandas DataFrames (`train_df` , `test_df`) into Hugging Face `Dataset` objects. These objects are optimized for use with Hugging Face models and trainers. The `text` and `label_encoded` columns are selected. Hugging Face إلى كائنات (`train_df` , `test_df`) Pandas DataFrames يحول. Face Dataset تم تحسين هذه الكائنات للاستخدام مع نماذج ومدربي . Face Dataset (.) يتم تحديد عمودي `text` و `label_encoded`

```

from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained(
    "distilbert-base-uncased"
)

def tokenize(batch):
    return tokenizer(
        batch['text'],
        padding='max_length',
        truncation=True,
        max_length=128
    )

```

Explanation:

- `AutoTokenizer.from_pretrained("distilbert-base-uncased")` : Loads the tokenizer associated with the `distilbert-base-uncased` model. A tokenizer is responsible for converting text into numerical input IDs that a model can understand. (مسؤول tokenizer الـ `distilbert-base-uncased` المرتبط بنموذج tokenizer يحمل الـ). عن تحويل النص إلى معرفات إدخال رقمية يمكن للنموذج فهمها.)
- `tokenize(batch)` function:
 - This function takes a batch of text and tokenizes it. (تأخذ هذه الدالة دفعة من النص وتقوم بترميزها.)
 - `batch['text']` : The text content to be tokenized. (محتوى النص المراد ترميزه.)
 - `padding='max_length'` : Pads the sequences to the maximum length specified, ensuring all inputs have the same length. (يحشو التسلسلات إلى أقصى طول محدد، مما يضمن أن جميع المدخلات لها نفس الطول.)
 - `truncation=True` : Truncates sequences longer than `max_length`. (يقتطع التسلسلات الأطول من `max_length`.)
 - `max_length=128` : Sets the maximum sequence length for tokenization. (يحدد الحد الأقصى لطول التسلسل للترميز.)

```

train_dataset = train_dataset.map(tokenize, batched=True)
test_dataset = test_dataset.map(tokenize, batched=True)

```

Explanation:

- `.map(tokenize, batched=True)` : Applies the `tokenize` function to all examples in the `train_dataset` and `test_dataset`. `batched=True` means the function will be applied to batches of examples, which is more efficient. This step converts the raw text in the datasets into tokenized numerical inputs (input IDs, attention masks, etc.) that the DistilBERT model expects. (على جميع الأمثلة في `tokenize` يطبق دالة) `train_dataset` و `test_dataset`. `batched=True` يعني أن الدالة ستطبق على دفعات من الأمثلة، وهو أكثر كفاءة. تحول هذه الخطوة النص الخام في مجموعات البيانات إلى مدخلات رقمية مرمزة (معرفات الإدخال، أقنعة الانتباه، إلخ) التي يتوقعها نموذج DistilBERT.)

```
train_dataset.set_format(
    type='torch',
    columns=['input_ids', 'attention_mask', 'labels']
)

test_dataset.set_format(
    type='torch',
    columns=['input_ids', 'attention_mask', 'labels']
)
```

Explanation:

- `set_format(type='torch', columns=['input_ids', 'attention_mask', 'labels'])` : Sets the format of the datasets to return PyTorch tensors and specifies the columns that should be returned when accessing elements from the dataset. It renames `label_encoded` to `labels` to conform to the expected input format for Hugging Face Trainer. (ويحدد PyTorch يحدد تنسيق مجموعات البيانات لإرجاع موثرات). الأعمدة التي يجب إرجاعها عند الوصول إلى العناصر من مجموعة البيانات. يعيد تسمية `label_encoded` إلى `labels` ليتوافق مع تنسيق الإدخال المتوقع لـ Hugging Face Trainer.)
 - **Note:** The notebook had an error here (`ValueError: Original column name label_encoded not in the dataset...`). This error indicates that the `label_encoded` column was not present or was already renamed. The correct approach is to ensure the `label_encoded` column exists before `set_format` or rename it explicitly before this step if needed. However, `set_format` itself can handle the renaming if the original column is present. (ملاحظة: كان هناك) لم يكن موجودًا `label_encoded` خطأ في الدفتر هنا. يشير هذا الخطأ إلى أن عمود أو تم إعادة تسميته بالفعل. النهج الصحيح هو التأكد من وجود عمود

أو إعادة تسميته صراحة قبل هذه الخطوة إذا `set_format` قبل `label_encoded` نفسها التعامل مع إعادة التسمية إذا كان `set_format` لزم الأمر. ومع ذلك، يمكن لـ (العمود الأصلي موجودًا)

```
from transformers import AutoModelForSequenceClassification,
TrainingArguments, Trainer
import numpy as np
import evaluate

num_labels = len(encoder.classes_)
model = AutoModelForSequenceClassification.from_pretrained(
    "distilbert-base-uncased", num_labels=num_labels
)

def compute_metrics(eval_pred):
    metrics = evaluate.load("accuracy")
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)
    return metrics.compute(predictions=predictions, references=labels)

training_args = TrainingArguments(
    output_dir="./results",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=3,
    weight_decay=0.01,
    evaluation_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
    metric_for_best_model="accuracy",
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    tokenizer=tokenizer,
    compute_metrics=compute_metrics,
)

trainer.train()
```

Explanation:

- `AutoModelForSequenceClassification.from_pretrained("distilbert-base-uncased", num_labels=num_labels)` : Loads a pre-trained DistilBERT model suitable for sequence classification. (مدرَّبًا مسبقًا ومناسبًا لـ DistilBERT يحمل نموذج.)
(.لتصنيف التسلسلات)
 - `"distilbert-base-uncased"` : Specifies the base model to load. (يحدد النموذج الأساسي للتحميل)
 - `num_labels=num_labels` : Configures the output layer of the model to match the number of unique classes in the dataset. (يقوم بتكوين طبقة الإخراج للنموذج). (لتتناسب مع عدد الفئات الفريدة في مجموعة البيانات)
- `compute_metrics(eval_pred)` function:
 - This function is used to calculate evaluation metrics during training. (تستخدم هذه الدالة لحساب مقاييس التقييم أثناء التدريب)
 - `evaluate.load("accuracy")` : Loads the accuracy metric from the `evaluate` library. (يحمل مقياس الدقة من مكتبة `evaluate`.)
 - `logits, labels = eval_pred` : Unpacks the predictions (logits) and true labels from the evaluation prediction object. (logits) يفك حزمة التنبؤات. (والتصنيفات الحقيقية من كائن تنبؤ التقييم)
 - `predictions = np.argmax(logits, axis=-1)` : Converts the logits (raw output from the model) into class predictions by taking the index of the maximum value along the last axis. (الإخراج الخام من النموذج) logits يحول إلى (ال). (تنبؤات فئة عن طريق أخذ فهرس القيمة القصوى على طول المحور الأخير)
 - `return metrics.compute(predictions=predictions, references=labels)` : Computes and returns the accuracy. (يحسب ويعيد الدقة)
- `TrainingArguments(...)` : Defines the hyperparameters and settings for training the model. (يحدد المعلمات الفائقة والإعدادات لتدريب النموذج.)
 - `output_dir="./results"` : Directory where model checkpoints and predictions will be saved. (الدليل الذي سيتم فيه حفظ نقاط فحص النموذج). (والتنبؤات)
 - `learning_rate=2e-5` : The initial learning rate for the optimizer. (معدل التعلم). (الأولي للمُحسِّن)

- `per_device_train_batch_size=16` , `per_device_eval_batch_size=16` : Batch sizes for training and evaluation. (أحجام الدفعات للتدريب والتقييم.)
- `num_train_epochs=3` : Total number of training epochs to perform. (العدد الإجمالي لعصور التدريب التي سيتم إجراؤها.)
- `weight_decay=0.01` : The weight decay to apply (if not zero, it's a form of L2 regularization). (إذا لم يكن صفرًا، فهو شكل من تضاؤل الوزن الذي سيتم تطبيقه. (L2 أشكال تنظيم.)
- `evaluation_strategy="epoch"` : Evaluation is performed at the end of each epoch. (يتم إجراء التقييم في نهاية كل عصر.)
- `save_strategy="epoch"` : Model checkpoints are saved at the end of each epoch. (يتم حفظ نقاط فحص النموذج في نهاية كل عصر.)
- `load_best_model_at_end=True` : Loads the best model (based on `metric_for_best_model`) at the end of training. (بناءً على) (يحمل أفضل نموذج) (في نهاية التدريب) (`metric_for_best_model`)
- `metric_for_best_model="accuracy"` : The metric to use to compare models and determine the best one. (المقياس الذي سيستخدم لمقارنة النماذج وتحديد) (الأفضل.)
- `Trainer(...)` : Initializes the Hugging Face `Trainer` object, which is a high-level API for training models. (وهو واجهة برمجة ، `Hugging Face Trainer` يهيئ كائن) (تطبيقات عالية المستوى لتدريب النماذج)
 - `model` , `args` , `train_dataset` , `eval_dataset` , `tokenizer` , `compute_metrics` : All the components required for training. (جميع المكونات) (المطلوبة للتدريب)
- `trainer.train()` : Starts the training process. (يبدأ عملية التدريب.)

```
trainer.evaluate()
```

Explanation:

- `trainer.evaluate()` : Evaluates the trained model on the `eval_dataset` (which is `test_dataset` in this case) and returns the evaluation metrics, including the loss. (في هذه الحالة `test_dataset` وهو) (`eval_dataset` يقيم النموذج المدرب على) (ويعيد مقاييس التقييم، بما في ذلك الخسارة)

- **Output shows:** `{'eval_loss': 1.2886754274368286}` . This is the validation loss after fine-tuning. (يظهر الإخراج: `{'eval_loss': 1.2886754274368286}` . هذه هي خسارة التحقق بعد الضبط الدقيق.)

8. Model Saving and Deployment Preparation

This section demonstrates how to save the trained traditional machine learning models (SVM and TF-IDF vectorizer) for later use, likely for deployment with Streamlit.

```
import joblib

joblib.dump(svm_model, 'svm_model.pkl')
joblib.dump(vectorizer, 'tfidf.pkl')
```

Explanation:

- `import joblib`: Imports the `joblib` library, which is efficient for serializing and deserializing Python objects, especially large NumPy arrays. (`joblib` ، المكتبة NumPy وهي فعالة لتسلسل وإلغاء تسلسل كائنات بايثون، خاصة مصفوفات الكبيرة.)
- `joblib.dump(svm_model, 'svm_model.pkl')`: Saves the trained `svm_model` to a file named `svm_model.pkl`. This allows the model to be loaded later without retraining. (هذا يسمح `svm_model.pkl` المدرب إلى ملف يسمى `svm_model` بحفظ.) بتحميل النموذج لاحقًا دون إعادة التدريب
- `joblib.dump(vectorizer, 'tfidf.pkl')`: Saves the fitted `TfidfVectorizer` to a file named `tfidf.pkl`. This is crucial because the same vectorizer used for training must be used for new data during inference. (`TfidfVectorizer` يحفظ) `vectorizer` هذا أمر بالغ الأهمية لأن نفس الـ `tfidf.pkl` المناسب إلى ملف يسمى `vectorizer` (المستخدم للتدريب يجب أن يستخدم للبيانات الجديدة أثناء الاستدلال)

```
model = joblib.load('svm_model.pkl')
vectorizer = joblib.load('tfidf.pkl')
```

Explanation:

- `joblib.load('svm_model.pkl')` : Loads the saved SVM model from the `svm_model.pkl` file. (المحفوظ من ملف SVM يحمل نموذج.)
- `joblib.load('tfidf.pkl')` : Loads the saved TF-IDF vectorizer from the `tfidf.pkl` file. (المحفوظ من ملف TF-IDF vectorizer يحمل.)

These loaded objects (`model` and `vectorizer`) would then be used in a Streamlit application to preprocess new input text and make predictions. (ستستخدم هذه الكائنات لمعالجة النص المدخل الجديد Streamlit بعد ذلك في تطبيق (`model` و `vectorizer`) المحملة (وإجراء التنبؤات).
